Unit II – Linear Data Structures Using Sequential Organization

Prof. Pranjal Pandit

Unit II – Linear Data Structures Using Sequential Organization

- Concept of sequential organization;
- Arrays as an ADT and array operations;
- merging of two arrays; storage representation and address calculation (row-major, column-major);
- multi-dimensional arrays.
- Ordered lists;
- Single-variable polynomial representation using arrays, polynomial as array of structures, polynomial addition and multiplication.
- Sparse matrices: representation, addition, simple and fast transpose, time–space trade-off.

Ordered lists

An **ordered list** is a collection of elements arranged in a specific sequence, where the **position (order) of each element matters**. Each element can be accessed by its index or position.

Characteristics of Ordered Lists

- Elements are stored in a defined order.
- Each element has a unique position (index).
- Duplicate elements are usually allowed.
- Elements can be accessed using their index.
- ccessed by its index or position.

Examples

Number List

10, 20, 30, 40, 50

Here:

First element = 10

Second element = 20

Third element = 30

If the order changes:

30, 10, 20, 40, 50

it becomes a different list because **order matters**.

Operations on an Ordered List

- **Insertion:** Add an element at a specific position.
- **Deletion:** Remove an element from a specific position.
- **Searching:** Find an element in the list.
- **Traversal:** Visit each element one by one.
- **Updating:** Change the value of an existing element.

Advantages

- Easy to access elements using their index.
- Suitable for sequential processing.
- Maintains the order of data.

Disadvantages

- Inserting or deleting elements in the middle may require shifting other elements.
- Fixed-size ordered lists (such as arrays) cannot grow dynamically.

An **ordered list** is a **logical concept (ADT - Abstract Data Type)**, while an **array** is a **physical implementation (data structure)** of an ordered list.

Key Differences

- **Array** is a **data structure**.
- **Ordered list** is an **Abstract Data Type (ADT)** or concept.
- An **array can be used to implement an ordered list**.
- An ordered list can also be implemented using **linked lists**, **dynamic arrays**, or other data structures.

Polynomial

A **polynomial** is a mathematical expression made up of variables, coefficients, and exponents.

General form:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x + a_0$$

where:

Coefficient (a): The numerical value multiplying the variable.

Exponent (n): The power of the variable.

Variable: Usually x.

Example

$$P(x) = 7x^5 + 4x^3 - 2x + 8$$

Single-variable polynomial representation using arrays

Polynomials are widely used in **Data Structures** to represent mathematical expressions such as:

$$P(x) = 5x^3 + 4x^2 - 2x + 7$$

A polynomial consists of **terms**, and each term has:

- **Coefficient** – the numerical value.
- **Exponent (Power)** – the power of the variable.

Example:

Term	Coefficient	Exponent
$5x^3$	5	3
$4x^2$	4	2
$-2x$	-2	1
7	7	0

Single Variable Polynomial Representation Using Arrays

In this method, only the **coefficients** are stored in an array. The **array index represents the exponent**.

Example

Polynomial:

$$P(x) = 5x^4 + 0x^3 + 2x^2 + 7x + 1$$

Array Representation

Exponent : 4 3 2 1 0

Coefficient : 5 0 2 7 1

Array:

```
int poly[5]={1,7,2,0,5};
```

Since index = exponent,

`poly[4]=5`

`Means :5x`^{4}

Index	Coefficient	Represents
0	1	1
1	7	7x
2	2	$2x^2$
3	0	$0x^3$
4	5	$5x^4$

Another Example

Polynomial

$$8x^3 + 6x^2 + 5x + 9$$

Stored as

Index	0	1	2	3
Value	9	5	6	8

Advantages

- Easy implementation.
- Fast access.
- Simple addition.

Disadvantages

- Wastes memory when many coefficients are zero.
- Not suitable for sparse polynomials.

Example

10x^100+2

Needs an array of size **101**, although only **2 coefficients** are non-zero.

Polynomial Representation Using Array of Structures

Why Do We Need an Array of Structures?

Suppose we represent the polynomial

$$P(x) = 7x^5 + 4x^3 - 2x + 8$$

using a normal array.

Exponent	5	4	3	2	1	0
Coefficient	7	**0**	4	**0**	-2	8

Notice that:

- Exponent **4** has coefficient **0**.
- Exponent **2** has coefficient **0**.

These zero values occupy memory even though they are not useful. If the polynomial is

$$10x^{100} + 5$$

then a normal array needs **101 elements**, although only **2 terms are non-zero**.

This wastes memory. To avoid this, we use an **array of structures**.

What is a Structure?

A **structure** is a user-defined data type that groups related variables together.

For a polynomial, each term has:

- Coefficient
- Exponent

So we define

```
struct term
{
    int coeff;
    int exp;
};
```

Here,

- coeff stores the coefficient.
- exp stores the exponent.

Each structure object represents **one term of the polynomial**. $P(x) = \mathbf{6x^4} + 3x^2 + 8$

(one object→ represents 1 Term)

Representing a Polynomial

Consider

$$P(x) = 6x^4 + 3x^2 + 8$$

There are only **3 non-zero terms**.

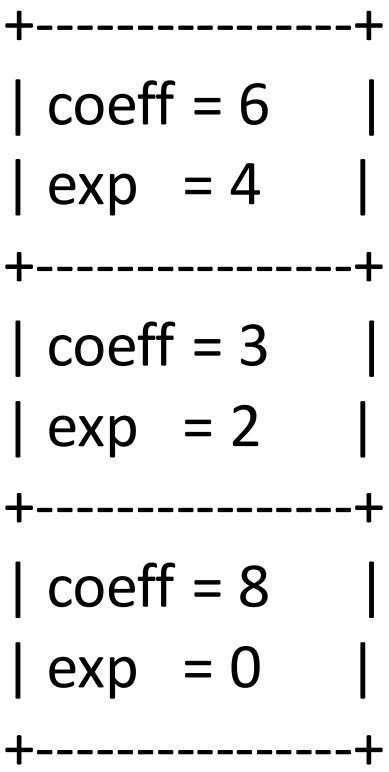
Instead of storing

Exponent	4	3	2	1	0
Coefficient	6	0	3	0	8

we store only the useful terms.

Coefficient	Exponent
6	4
3	2
8	0

Memory representation



```
#include<iostream>
using namespace std;
struct term
{
   int coeff;
   int exp;
};
int main()
{
   term p[3]=                //created 3 objects of structure
   {
      {6,4},
      {3,2},
      {8,0}
   };
   for(int i=0;i<3;i++)
   {
      cout<<p[i].coeff<<"x^"<<p[i].exp<<endl;
   }
   return 0;
}
```

Output

```
6x^4
3x^2
8x^0
```

Example 2

Polynomial

$$P(x) = 9x^6 - 5x^3 + 2x - 7$$

Step 1: Identify terms

Term	Coefficient	Exponent
$9x^6$	9	6
$-5x^3$	-5	3
2x	2	1
-7	-7	0

Step 2: Create Array of Structures

```
term p[4]=          //4 objects as 4 non zero terms
{
     {9,6},
     {-5,3},
     {2,1},
     {-7,0}
};
```

Example 3

Polynomial

$$12x^8 + 4x^5 - 3x^2 + 9x - 11$$

Structure Representation

Coefficient	Exponent
12	8
4	5
-3	2
9	1
-11	0

```
term p[5]= {
{12,8},
 {4,5},
 {-3,2},
 {9,1},
 {-11,0}
};
```

Accessing Individual Terms

Suppose

term p[3]=

```
{
   {6,4},
   {3,2},
   {8,0}
};
```

Then p[0].coeff

Returns 6

p[0].exp

Returns 4

p[1].coeff

Returns 3

p[2].exp

Returns 0

Displaying the Polynomial

```
for(int i=0;i<3;i++)
{
   cout<<p[i].coeff<<"x^"<<p[i].exp;
}
```

Output

6x^4

3x^2

8x^0

A more readable display is:

6x^4 + 3x^2 + 8

(The constant term, where exponent = 0, is usually printed without x^0.)

Advantages

1. Saves Memory

Only non-zero terms are stored.

Example

$$100x^{100} + 2$$

Normal array

101 elements

Array of structures

Only 2 structures

2. Suitable for Sparse Polynomials

Sparse polynomial

$$9x^{50} + 7x^3 + 2$$

Only three structures are needed.

3. Easier Polynomial Operations

Operations such as:

- Addition
- Multiplication
- Evaluation

are simpler because only non-zero terms are processed.

Disadvantages

- Accessing a term with a specific exponent requires searching through the structures.
- Slightly more complex than a simple array.
- Terms should usually be stored in descending order of exponents for efficient processing.

Practice Questions (Try Yourself)

Question 1

Represent the polynomial using an array of structures:

$$8x^7 + 6x^4 - 5x + 10$$

Question 2

Represent the polynomial:

$$15x^9 - 2x^6 + 7x^3 + 4$$

Question 3

Represent the polynomial:

$$11x^5 - 8x^4 + 3x^2 - 9$$

Answer 1

```
term p[4] =
{
     {8,7},
     {6,4},
     {-5,1},
     {10,0}
};
```

Coefficient	Exponent
8	7
6	4
-5	1
10	0

Coefficient	Exponent
15	9
-2	6
7	3
4	0

Answer 2

```
term p[4] =
{
    {15,9},
    {-2,6},
    {7,3},
    {4,0}
};
```

Polynomial Addition

Two polynomials can be added only by adding terms with **equal exponents**.

Example

$$P(x)=5x^3+4x^2+2x+1$$

$$Q(x)=3x^3+2x^2+8$$

Arrange

$5x^3+4x^2+2x+1$

$+\quad 3x^3+2x^2+0x+8$

$8x^3+6x^2+2x+9$

Result

$$R(x)=8x^3+6x^2+2x+9$$

Using Arrays

P

Index : 0 1 2 3

Value : 1 2 4 5

Q

Index : 0 1 2 3

Value : 8 0 2 3

Add corresponding elements

R[0]=1+8=9

R[1]=2+0=2

R[2]=4+2=6

R[3]=5+3=8

Final array

9 2 6 8

Represents

$8x^3+6x^2+2x+9$

Polynomial Multiplication

- Polynomial multiplication is the process of **multiplying every term of one polynomial by every term of another** polynomial.
- After multiplication, **like terms** (terms having the same exponent) **are combined by adding their coefficients.**

Rules of Polynomial Multiplication

When multiplying two terms:

- **Multiply the coefficients.**
- **Add the exponents of the variables.**

For example:

$(3x^2) \times (4x^3)$

$\text{Coefficient} = 3 \times 4 = 12$

$\text{Exponent} = 2 + 3 = 5$

$\text{Result} = 12x^5$

Example 1

Multiply

$$P(x) = 2x + 3$$
$$Q(x) = x + 4$$

Step 1: Multiply the first term of P with every term of Q

$2x \times x = 2x^2$

$2x \times 4 = 8x$

Step 2: Multiply the second term of P with every term of Q

$3 \times x = 3x$

$3 \times 4 = 12$

Step 3: Write all terms $2x^2 + 8x + 3x + 12$

Step 4: Combine like terms $8x + 3x = 11x$

Final Answer

$$2x^2 + 11x + 12$$

Practice

Example 2

Multiply

$$P(x) = x^2 + 2$$
$$Q(x) = 3x + 4$$

Example 3

Multiply

$$P(x) = 2x^2 + 3x + 1$$
$$Q(x) = x + 5$$

Example 4

Multiply

$$P(x) = x^2 + 2x + 1$$
$$Q(x) = x^2 + x + 2$$

Answer 2: $3x^3 + 4x^2 + 6x + 8$

Answer 3: $2x^3 + 13x^2 + 16x + 5$

Answer 4: $x^4 + 3x^3 + 5x^2 + 5x + 2$

Polynomial Multiplication Using Array of Structures

Suppose

Polynomial P

Represents

$$3x^2 + 2x + 1$$

Polynomial Q

Represents

$$2x + 4$$

Multiplication (each term of P * each term of Q)

Term from P	Term from Q	Coefficient	Exponent	Result
$3x^2$	2x	6	3	$6x^3$
$3x^2$	4	12	2	$12x^2$
2x	2x	4	2	$4x^2$
2x	4	8	1	8x
1	2x	2	1	2x
1	4	4	0	4

Now combine like terms.

$12x^2 + 4x^2 = 16x^2$

$8x + 2x = 10x$

Final polynomial

$$6x^3 + 16x^2 + 10x + 4$$

C++ Code

```
for(i=0; i<m; i++)
{
   for(j=0; j<n; j++)
   {
      result.coeff = P[i].coeff * Q[j].coeff;
      result.exp = P[i].exp + Q[j].exp;

      // Check if exponent already exists
      // If yes, add coefficient
      // Otherwise, insert new term
   }
}
```

Time Complexity

If:

First polynomial has **m** terms.

Second polynomial has **n** terms.

Then:

Multiplication: $O(m \times n)$

Combining like terms: depends on the method used (often **$O((m \times n)^2)$** with a simple search, or better with optimized data structures).